

Chapter 1

Week 9 — Advanced OOP & C# Conventions

ENEL712 Embedded Systems Design — Lecture Week 9

Summary

Advanced OOP concepts and C# conventions: naming styles, static behaviour, inheritance, polymorphism (overloading + overriding), and design constraints (sealed, abstract, enums).

Topics Covered

- Naming and language conventions
- Static behaviour
- Inheritance and reuse
- Polymorphism and method overriding
- Design constraints and encapsulation
- Enumerations

1.1 Naming and Language Conventions

1.1.1 Pascal Case

- Used for classes, interfaces, structs, and public members (methods, properties, events).
- Unifies the public contract under one readable style.

1.1.2 Camel Case

- Used for private/internal fields (often `_` prefix), local variables, and delegate instances.
- Distinguishes implementation details from the public interface.

1.1.3 Identifier rules

- C# uses neither `snake_case` nor `SCREAMING_SNAKE_CASE`.
- Identifiers start with a letter or underscore; cannot contain two consecutive underscores (reserved).

1.2 Static Behaviour

- Static methods are utility functions independent of instance state — bound at compile time.
- Static data members are shared across all instances; initialised once when the class loads.
- `const` fields are implicitly static and immutable (compile-time constants).
- `readonly` is a runtime constant — set at declaration or in the constructor.

1.3 Inheritance and Reuse

- Syntax: `class DerivedClass : BaseClass` — single inheritance only.
- `base` keyword calls the base constructor and accesses hidden/overridden members.
- `protected` members are accessible to the class itself and derived classes.

1.4 Polymorphism and Method Overriding

- Overloading (compile-time polymorphism): same name, different parameter list.
- Virtual methods are optionally overridable; runtime decides the version called (dynamic polymorphism).
- Abstract methods have no base implementation and must be overridden by derived classes.
- Override methods must use `override` and exactly match the base signature.

1.5 Design Constraints and Encapsulation

- `public/private/protected` control visibility; C# also has `internal` and `protected internal`.
- Sealed class: no further inheritance allowed.
- Sealed (overridden) method: no further override allowed.
- Sealing helps performance optimisation and prevents inappropriate extension.

1.6 Enumerations

- Assign meaningful names to integer values (e.g., `enum Season {Spring, Summer, Autumn, Winter};`).
- Improve readability and type safety, particularly in switch-case.
- Default underlying type is `int`.

1.7 Use Cases & Applications

- Following Microsoft naming conventions makes your code blend into the rest of the .NET ecosystem and look familiar to other engineers.
- static utility classes are a clean home for pure helpers that don't keep state.
- Inheritance is best for is-a relationships (a Manager is an Employee). Use composition (has-a) when behaviour just needs to be combined.
- sealed is a deliberate design statement: 'this hierarchy is closed'. Useful in libraries to prevent breaking changes.
- enum makes finite, named state spaces explicit instead of relying on magic integers.

1.8 Naming Convention Quick Reference

Element	Convention	Example
Class / interface / struct	PascalCase	MotorController
Public member (method, property)	PascalCase	Open(), Speed
Private field	camelCase or _camelCase	_speedRpm
Local variable / parameter	camelCase	rpmTarget
Constant	PascalCase	MaxSpeed

1.9 Worked Example: Method Overriding

```
public class Animal
{
    public virtual string Speak() => "...";
}

public class Dog : Animal
{
    public override string Speak() => "Woof";
}

public class Husky : Dog
{
    // 'sealed override' freezes Speak() -- no further customisation allowed.
    public sealed override string Speak() => "Awooo";
}
```

1.10 When to Use What

Need	Use
Behaviour shared by several types	abstract base class with virtual methods
Pure utility, no instance state	static class with static methods
Multiple capabilities mixed in	interface (single inheritance limits classes)
Compile-time constant	const
Runtime constant settable in constructor	readonly
Finite, named integer set	enum

Key Definitions

Pascal Case

Capitalise the first letter of each word, e.g., MyClassName.

Camel Case

First letter lowercase, subsequent words capitalised, e.g., myLocalVariable.

Static Member

Member belonging to the class itself rather than instances.

Inheritance

Mechanism letting one class extend another's functionality.

Polymorphism

Many forms — different objects respond to the same call differently.

Method Overloading

Compile-time polymorphism via different parameter signatures.

Virtual Method

Base method that may be overridden in derived classes (runtime polymorphism).

Abstract Method

Method without base implementation that must be overridden.

Encapsulation

Bundling data and methods and restricting external access.

Sealed Class

Class explicitly prevented from being inherited.

readonly

Field that can be assigned only at declaration or in a constructor; runtime equivalent of const.

abstract Class

Class that cannot be instantiated directly; may declare abstract methods that subclasses must implement.

Sealed Override

Marks an override as final — no further class may override that method again.

Composition over Inheritance

Design preference for combining objects (has-a) instead of subclassing (is-a) when both could work.

Sources

- Lecture Week 9.pdf